TPO 2

Para el segundo trabajo práctico, se deberán entregar los siguientes ejercicios de clase:

- Clase 8 Ejercicio 7
- Clase 9 Ejercicio 6
- Clase 10 Ejercicio 2
- Clase 10 Ejercicio 3
- Ejercicio árbol

Debajo, a modo de anexo, se colocan los enunciados para evitar confusiones (son los mismos que fueron propuestos para cada clase) **sumado el ejercicio del TDA árbol que no está en los ejercicios de clase**.

Cada grupo debe entregar un proyecto por ejercicio, consolidado en **un único archivo comprimido "TPO2-GRUPO X.zip"** (X es el nombre/número del grupo) que contiene las 5 carpetas que corresponden a los 5 proyectos solicitados.

La entrega la realiza un único integrante del grupo (enviar un solo correo), poniendo en copia a todos sus compañeros, por mail a wgomez@uade.edu.ar con el asunto **TPO2-GRUPO X-2026-2C.**

La fecha de entrega es 27/10/2026.

Obs: El archivo puede ser bloqueado por razones de seguridad al momento de enviar el correo, esto se debe a los ejecutables que se encuentran dentro de las carpetas "bin" de los proyectos, en caso de ser necesario, se puede vaciar el contenido de dichas carpetas.

Como última instancia, se puede realizar la entrega mediante teams, siempre respetando el formato indicado previamente.

Clase 8 - Ejercicio 7

Implemente un TDA Dictionary (diccionario múltiple) utilizando un ABB para la estructura que almacena los valores (en lugar de una lista o arreglo).

Clase 9 - Ejercicio 6

Cree un programa que solicite al usuario una serie de inputs numéricos que se detienen con el símbolo “.” Los elementos se agregan a un AVL, se muestra el árbol generado por niveles y se genera un arreglo con los elementos ordenados de menor a mayor que también se muestra por pantalla.

Clase 10 - Ejercicio 2

Implementar el TDA Grafo definido en el ejercicio anterior, utilizando matriz de adyacencia (implementación estática).

Sobra esta implementación agregar los siguientes métodos:

1. Dado un vértice v, calcular el mayor de los costos de las aristas salientes.
2. Dado un vértice v, escribir un método que permita obtener el conjunto de los Predecesores del vértice.

Clase 10 - Ejercicio 3

Implementar el TDA Grafo definido en el ejercicio 1, utilizando listas de adyacencia (implementación dinámica).

Sobra esta implementación agregar los siguientes métodos:

1. Escribir un método que permita obtener el conjunto de los vértices aislados en G. Se define que un vértice v es aislado si v no tiene aristas entrantes ni salientes.
2. Escribir un método que permita obtener el conjunto de todos los vértices puente entre v1 y v2. Se define que un vértice p es puente entre dos vértices o y d, si hay una arista que comienza en o y termina en p y otra que comienza en p y termina en d.

Ejercicio árbol

El grupo debe implementar el TDA Árbol N-ario genérico en Java y resolver un conjunto de operaciones sobre él usando recursión.

Un árbol n-ario (o árbol general) es una estructura jerárquica con una única raíz, donde cada nodo guarda un valor y tiene entre 0 y n hijos, sin límite fijo. Cada nodo, salvo la raíz, tiene exactamente un padre y no hay ciclos. Los hijos de un nodo mantienen el orden en que fueron agregados. Se debe utilizar la lista implementada en la cátedra, no está permitido el uso de estructuras de Java (java.util). A continuación, se muestra la interfaz:

```
public interface Tree<T> {

    // --- Construcción y modificación ---
    void agregarRaiz(T valor);
    void agregarHijo(T padre, T hijo);
    void eliminar(T valor);

    // --- Consultas sobre el árbol ---
    boolean esVacio();
    T raiz();
    int tamanio();
    int altura();
    boolean contiene(T valor);

    // --- Consultas sobre un nodo ---
    T padre(T valor);
    List<T> hijos(T valor);
    int grado(T valor);
    boolean esHoja(T valor);
    int profundidad(T valor);

    // --- Recorridos ---
    List<T> preorden();
    List<T> postorden();
    List<T> porNiveles();
}
```

Operación	Precondición	Resultado
`agregarRaiz(v)`	Árbol vacío, `v` no nulo	`v` pasa a ser la raíz
`agregarHijo(p, h)`	`p` existe, `h` no existe ni es nulo	`h` se agrega como último hijo de `p`
`eliminar(v)`	`v` existe	Se elimina `v` con todo su subárbol; si es la raíz, el árbol queda vacío
`esVacio()`	—	`true` si no tiene nodos
`raiz()`	Árbol no vacío	Valor de la raíz
`tamanio()`	—	Cantidad total de nodos
`altura()`	—	Cantidad de aristas del camino más largo raíz–hoja: −1 si está vacío, 0 si sólo hay raíz
`contiene(v)`	—	`true` si `v` está en el árbol
`padre(v)`	`v` existe	Valor del padre, o `null` si `v` es la raíz
`hijos(v)`	`v` existe	Copia de la lista de hijos, en orden de inserción
`grado(v)`	`v` existe	Cantidad de hijos de `v`
`esHoja(v)`	`v` existe	`true` si `v` no tiene hijos

`profundidad(v)`	`v` existe	Aristas desde la raíz hasta `v` (la raíz tiene 0)
`preorden()`	—	Nodo, luego cada hijo de izquierda a derecha
`postorden()`	—	Cada hijo de izquierda a derecha, luego el nodo
`porNiveles()`	—	Nivel 0, nivel 1, …, cada nivel de izquierda a derecha

Se debe tener en cuenta lo enunciado para implementar el TDA, para ello:

- **Clase nodo**. Implementen TreeNode<T> con un atributo valor y una List< TreeNode <T>> hijos. Pueden agregar una referencia al padre.
- **Clase implementación del TDA**. DinTree<T> implements Tree<T>, con la raíz como atributo y el tamaño mantenido en un contador (no recalculado en cada llamada).